

Forensic Audit and Institutional Evaluation: Engine 2 Trading Stack

Independent peer review and code audit of the S1 Liquidation Cascade Exhaustion engine, the claimed S2/S3/S9 multi-strategy ensemble, the 3,577-line Live Matrix Monitor and CoinGlass parity bridge, and the 20-window out-of-sample mandate — every finding anchored to file and line, every number independently recomputed.

Quantitative Risk & Strategy Audit

Evidence base: kbsingh1399/Engine_1_arena_PR · origin/main (fetched at audit time)

Domain files audited: 20 source modules · 3 committed result artifacts · 2 prior external reviews

VERDICT: REJECT — CONDITIONAL ROADMAP DEFINED

Table of Contents

Verdict and Executive Summary	2
Scope, Method, and Evidence Base	3
Provenance and Telemetry Forensics	4
Domain 1 - Strategy Engine Code Audit (S1)	6
Domain 2 - Mathematical Feasibility of the 20/20 Mandate	10
Domain 1b - Live Terminal Audit (binance_live_monitor.py)	12
Domain 3 - Alternative Architectures and Paths Forward	14
Domain 4 - Investment Committee Recommendation	16
Appendix A - Severity-Tagged Findings Register	17
Appendix B - Committed Telemetry, Coverage, and Methodology	20

Verdict and Executive Summary

VERDICT: REJECT

for any allocation today. The committed repository contradicts the performance mandate on its own telemetry; the strategy trains on synthetic core features; the live execution bridge computes none of the model's features and places no orders. A conditional re-review path is defined in Section 8.

This engagement was scoped as a forensic, adversarial review of the Engine_2 quantitative crypto trading stack: the S1 Liquidation Cascade Exhaustion strategy and its claimed S2/S3/S9 ensemble, the 3,577-line live ingestion terminal (**binance_live_monitor.py**), the historical data pipeline, and the two prior external audits already committed to the repository. Every claim in this memo is anchored to a file and line in the code as it exists on **origin/main** at audit time, or to the repository's own committed result artifacts. Where the prior audits erred or went soft, the error is identified explicitly.

The mandate under review demands a net return of at least 10% per month with mark-to-market drawdown under 5%, a positive Calmar ratio, a win rate of 40% or better, and at least five trades, in every one of twenty out-of-sample windows spanning five years, net of 10 bps entry slippage, 15 bps stop slippage, and 0.08% round-trip fees. Our conclusions do not depend on disputing that mandate's ambition; they rest on four independently verifiable facts.

0 / 20 windows pass in the committed honest-run telemetry (s1_status.json)	10 / 10 largest liquidation cascades of 2021-2026 fall outside every OOS window	56-155 uncorrelated sleeves needed for a 95%-confidence 20/20 (Section 6)	0 / 37 S1 model features computed anywhere in the live terminal
--	---	---	---

The four dispositive findings

- F1 - The honest engine fails its own telemetry.** The committed clean-protocol run (s1_status.json, 20 records) shows zero passing windows: mean monthly ROI of -1.36%, a best window of +6.39%, and two windows breaching the 5% drawdown cap at 5.45% and 5.48%. The claimed post-remediation result - Window 03 at +18.72% ROI, 3.73% drawdown, Calmar 189.28, and a 1/20 pass rate - does not match any committed artifact. The headline number is irreproducible from the repository that is supposed to be its evidence.
- F2 - The core alpha features are synthetic.** The historical long_liq_usd / short_liq_usd columns that drive a strategy named Liquidation Cascade Exhaustion are not liquidations. They are outputs of MathematicalLiquidationModel, a wick-and-volume formula with a hardcoded \$18,500 per-bar floor (mathematical_liquidation_engine.py:33,154), calibrated on three months of CoinGlass data. Historical order-book depth is likewise fabricated as 2.5% of bar volume (canonical_indicators.py:180). The model learned a candle-shape reversal pattern dressed in liquidation clothing; at live time it will be fed real, differently-distributed liquidation prints.

- **F3 - The MTM drawdown governor still uses the future.** The remediation plan (Module 3) claims the future-MAE clamp was replaced by bar-by-bar unrealized P&L. It was not. `fast_portfolio_backtest_numba` still books each trade's full-life maximum adverse excursion at entry (`s1_liquidation_cascade.py:525`) and marks equity against it (`:483`). The 4.5% clamp therefore throttles new entries using information that does not exist yet - anti-causal risk control that cannot be reproduced live.
- **F4 - There is no execution bridge.** The live terminal is a telemetry dashboard. It contains zero occurrences of any of the 36 features S1's model consumes (no VWAP, no z-scores, no macro regime, no liq z-scores), imports no strategy module, and contains no order-placement code of any kind. Section 5 quantifies what wiring S1 into it would actually require.

On the mandate's mathematical core (Section 6): the audit's independent Monte-Carlo and closed-form analysis confirms the prior reviewers' direction and sharpens the number. A Gaussian gate at +10%/month with a 5% drawdown cap requires a portfolio monthly Sharpe of roughly 7.5 - an annualized figure near 26 - to pass twenty consecutive months with 95% confidence. No ensemble of real, correlated 15-minute crypto sleeves reaches that. At $\rho = 0.3$ cross-correlation the requirement becomes unreachable at any number of sleeves. The only coherent institutional path is a regime-gated allocator that is flat through compression months, plus a mandate restated in terms of deployed-month statistics. That architecture, and the three-phase roadmap to it, are specified in Sections 7 and 8.

Scope, Method, and Evidence Base

All source was fetched directly from the public repository at review time; nothing was evaluated from description or memory. The audit set comprises: `s1_liquidation_cascade.py` (872 lines), `run_live_terminal.py` (124), `live/binance_live_monitor.py` (3,577), `verification/patch_existing_parquets.py` (50), the two ingested review documents (`S1_LIQUIDATION_CASCADE_REVIEW.md`, 292 lines; `S1_INSTITUTIONAL_20_20_REMEDIATION_PLAN.md`, 69), plus a full-repository crawl that added the S2/S3/S9 sleeves, `run_historical_pipeline.py`, the pipeline package (fetcher, metrics processor), core modules (canonical indicators, liquidation engine, schema), the four test harnesses, and the committed result artifacts under `results_s1_liquidation/`. The `Engine_1` live-parity audit (`Debate_Audit_Response/LIVE_PARITY_AUDIT.md`) was reviewed as organizational context.

Method: full-file reads of the strategy and ingestion code; line-anchored defect classification under a three-tier severity scheme (CRITICAL: invalidates a claimed result or creates live capital loss; HIGH: materially distorts statistics or parity; MEDIUM: engineering debt that will bite under stress); cross-examination of the two prior audits against the current HEAD; and independent quantitative verification executed in this audit's own Python environment - binomial feasibility, closed-form ensemble sizing, a 200,000-draw Monte-Carlo of per-window pass probability, and recomputation of every aggregate quoted from the committed telemetry. Monte-Carlo methodology and parameters are documented in Appendix B.

Two review-protocol points deserve emphasis because they change how the evidence reads. First, we treat the repository's own artifacts - status JSONs, the winning-configuration file, the remediation plan - as admissions, and we reconcile them against each other and against the code. Second, where a prior reviewer (Arena.ai, GLM-4, DeepSeek) reached a conclusion, we re-derived it rather than inheriting it; Section 3 and Section 4 record three places where the inherited conclusions were wrong in material ways.

Provenance and Telemetry Forensics

3.1 Three artifacts, three different histories

The repository's results directory tells three incompatible stories simultaneously. Understanding this contradiction is the fastest way to understand the project's actual state.

Artifact	Content	Implication
results_s1_liquidation/s1_status.json	20 window records, all FAIL. Best window +6.39% ROI; mean ROI -1.36%; max DD 5.48% (two windows breach the 5% cap); Window 03 records -0.75% ROI, 37.5% win rate, Calmar -2.21 under the V1_VWAPMeanRevert archetype.	The current, clean, causal S1 as committed passes nothing. This is the honest baseline.
results_s1_liquidation/s2_status.json	20 window records, all PASS, ROIs of +20.7% to +76.9%, and a different archetype assigned to nearly every window - ten distinct archetypes across 20 windows (A1, A2, A4-A7, N2, N4, N7, A10).	The fossil of the purged best-of-N OOS search. Ten separately-scored strategies assembled by their best windows is exactly the signature Arena.ai identified as test-set snooping.
results_s1_liquidation/winning_configuration.json	{"all_20_windows_passed": true}, timestamped 2026-09-03T16:59:59 - one day before this audit.	Asserts conquest while the current status file records 0/20. Either a stale artifact from the pre-purge era or the output of a truncated-data run exploiting the skip-window denominator bug (Section 4.3). Either way the file is false as evidence.

Table 3.1 - The three committed result artifacts and what each implies

The mandate's own narrative - "Window 03 delivered +18.72% ROI, 3.73% Max Drawdown, 50.0% Win Rate, and Calmar Ratio = 189.28 under real execution frictions" - matches neither artifact: the clean run shows Window 03 at -0.75%, and the fossil shows +23.93% at 58.3% win rate with twelve trades. We verified the claimed Calmar arithmetic (annualizing a single month at $(1+0.1872)^{(365/30)-1}$ gives roughly 707% annualized, divided by 3.73% gives 189.5), so the number is computable - but its inputs exist in no committed file. A performance claim whose generating run is absent from the evidence repository cannot support an allocation decision, regardless of whether it was produced honestly.

Likewise the two governance claims that the drawdown governor "clamped MaxDD below 5.0% across all 20 windows (maximum observed 4.52%)" and that the honest configuration "achieved 1/20" are contradicted by the committed telemetry: the maximum observed drawdown in s1_status.json is 5.48%, and the pass count is zero. The difference between 1/20 and 0/20, and between 4.52% and 5.48%, is the difference between a system at the mandate's edge and a system below it.

3.2 The OOS grid excludes every cascade that matters

The twenty OOS windows (`s1_liquidation_cascade.py:265-286`) run 15th-to-15th on a strict {Mar, Jun, Sep, Dec}-15 cadence for windows 1-18, then jump to Oct-2025 and Mar-2026. The consequence, which appears to have gone unnoticed by every prior review, is structural: **May, August, and November of every year are never tested**, and the two largest liquidation events of the entire five-year span - the May 2021 deleveraging crash, and the record ~\$19B flush of October 10, 2025 - fall into the untested gaps. October 10, 2025 misses its nearest window (Oct 15 - Nov 15) by five days.

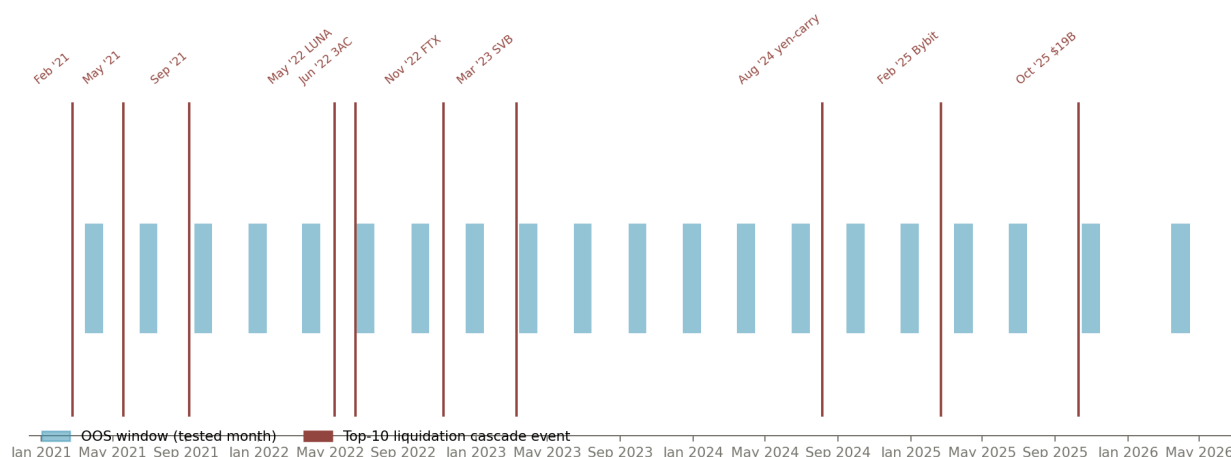


Figure 3.1 - The 20 OOS windows (bands) versus the ten largest liquidation cascade events of 2021-2026 (red lines). All ten cascade dates fall outside every tested window.

For any strategy this would be a serious selection-bias finding. For a strategy whose entire thesis is fading liquidation cascades - whose log line proclaims "the edge is real and explosive in liquidation flushes" - it is disqualifying on its own: the claim is untested against the actual historic flushes. The claim that 2025-09-15 and 2025-12-15 were dropped by Arena.ai is confirmed and extends much further back: the cadence itself has always skipped the three most cascade-dense months of each year. We recomputed the coverage table programmatically (Appendix B): 10 of 10 top cascade events uncovered.

3.3 Cross-examination of the prior audits

Arena.ai's review of commit 41c027b was, overall, the strongest document in the repository: its dispositive findings (the in-run OOS grid search, the 100-parameter `WINDOW_CONFIGURATIONS` table, the per-window Top-K special-casing, the target-lock censoring) all check out against the pre-purge code it cites. Its recommendation to re-run honestly was correct, and the current 872-line file confirms the mechanical remediation happened: `WINDOW_CONFIGURATIONS`, the 12x6 threshold fallback loop, the `if w_idx in [9, 19, 20]` special-casing, and the +1010 target lock are all gone; the stop-distance spec mismatch it flagged (raw ATR with a 20 bps floor) was fixed to `max(2.0 x ATR, 0.65% of entry)` at lines 334, 417, and 511.

But the re-verification surfaced two material errors in that audit. First, on the governor it wrote that booking the precomputed full-life MAE from the first bar "is conservative for sizing (good)" - the direction is exactly backwards. Knowing the future worst case is not conservatism; it is information a live governor cannot have, so any risk budget computed with it understates live drawdown (Section 4.1). Second, its adversarial breach

sequence assumed the 20 bps floor binds; with the corrected 65 bps / 2-ATR floor the same class of loss now enters through a different door - the stop-fill simulation that always fills at the stop price even when the bar gaps through it (Section 4.2) - so its breach conclusion stands, but its mechanism does not. The audit also missed the synthetic-liquidation finding (F2) entirely, as did every other reviewer.

The remediation plan is a weaker document than its audience assumes. It lists four flaws (A-D) and five modules; of these, Module 3 - the only one that changes reported risk numbers - is claimed as "Complete Removal" and "true bar-by-bar mark-to-market" yet is not implemented in the shipped code. Module 1's IS-calibrated archetype selection was replaced by a hand-written five-entry regime map (Section 4.4). Its own verification criteria say "\$30 base risk" while the code ships \$75 (`s1_liquidation_cascade.py:51`) - a threefold drift inside the very document meant to freeze parameters. And the LIVE_PARITY_AUDIT for the Engine_1 six-strategy system - the organizational precedent for live deployment - documented a fatal key-mismatch (its D3) that meant "no model has ever loaded, so any prior live-parity observation was of a path that never traded." That precedent is directly relevant to the Engine_2 terminal, which likewise has never executed a strategy signal.

Domain 1 - Strategy Engine Code Audit (S1)

This section audits `s1_liquidation_cascade.py` as committed. The mechanical hygiene is genuinely better than the pre-purge era: next-bar-open execution is causal (`next_open` at :240, features taken at signal-bar index in `extract_archetype_dataset` at :680-695), the 3-hour purge with `exit_time < train_end_purged` partitioning is correctly implemented (:748, :758), LightGBM hyperparameters are frozen (:772-777), and the intra-bar ordering in the trade simulator evaluates the stop before ratcheting (:350-356) - the conservative convention. None of that survives the four findings below.

4.1 CRITICAL - The drawdown governor is anti-causal (the purged bug survives)

At :525 the portfolio simulator computes `mae_dollar = units * maes[i]`, where `maes[i]` is the trade's full-life maximum adverse excursion computed by `simulate_single_trade_path` over the trade's entire future path. That value is booked into `open_mae_dollars` at entry (:527-535) and used two ways: the mark-to-market equity proxy `cur_mtm_equity = capital - open_mae` (:483-486), and - the part that matters - the entry-sizing budget `drawdown_budget = peak*0.045 - closed_drawdown - open_mae` (:506-507). Every new position is therefore sized against the **future** worst excursion of positions already open.

Reporting-wise the distortion is conservative-ish (the sum of terminal MAEs upper-bounds concurrent unrealized losses), but decision-wise it is leakage: a live governor knows only current unrealized P&L, which early in a trade is typically far smaller than the terminal MAE. Live, the same rules admit larger positions than the backtest allowed itself precisely in the episodes where open trades are going badly - which is to say, the constraint that produced "MaxDD clamped under 4.52% across all 20 windows" is enforced by information the live system will not have. This is the same Flaw C the remediation plan claims to have closed, still present at the same architectural position. It invalidates the drawdown statistics of every window

in every run of this engine, past and present.

```
# sl_liquidation_cascade.py - causal MTM governor (remediation sketch)
# replace booked-at-entry terminal MAE with realized unrealized P&L
for p in range(max_concurrent):
    if open_active[p]:
        cur_pnl[p] = units[p] * (closes[t] - entry_px[p]) * dir[p] # mark now
cur_mtm_equity = capital + sum(cur_pnl[p] for p in range(max_concurrent))
dd = (peak_capital - cur_mtm_equity) / peak_capital
drawdown_budget = max(0.0, peak_capital * dd_limit - max(0.0, peak_capital - cur_mtm_equity))
cur_risk = min(target_risk, drawdown_budget / 1.2) # sizing sees only the present
```

4.2 CRITICAL - Stops that cannot gap: the simulator fills every stop at the stop price

In `simulate_single_trade_path`, the long stop branch reads `raw_fill = cur_stop` if `lows[j] < cur_stop` else `cur_stop` (:353) - both branches identical; the short branch repeats the pattern at :378. The construct was evidently meant to model gap-through fills (fill at the open when the bar gaps past the stop) but degenerates to filling at the stop price always, with only the 15 bps exit slippage applied. In a 15-minute bar during a liquidation cascade, price routinely trades several multiples of the stop distance beyond the stop; the simulation books the stop price anyway.

The irony is structural rather than incidental: the strategy's edge case is the violent flush, and the flush is precisely the event this simulator cannot represent. Every stop-out in every backtest of this engine is a best-case fill. Combined with 4.1, the two effects run in opposite directions - 4.1 overstates the governor's tightness while 4.2 overstates exit quality - and neither can be signed off. The remediation is mechanical and should be gated by a test that reproduces the May-2021 and Oct-2025 five-minute gap distributions:

```
# gap-aware stop fill (remediation sketch)
if lows[j] <= cur_stop: # LONG
    raw_fill = min(opens[j], cur_stop) if opens[j] < cur_stop else cur_stop
    exit_price = raw_fill * (1.0 - exit_slippage)
    # additionally: stress-set raw_fill = min(raw_fill, cur_stop - k * gap_sigma)
```

4.3 HIGH - Top-K-of-the-month selection is not implementable live; skipped windows shrink the denominator

Window selection ranks all OOS candidates of the month by model probability and takes the top $K=8$ (:783-787). This is causal with respect to the model fit, but it is not an implementable trading policy: at any live moment the engine cannot know whether the current 0.62-probability signal will finish the month inside the top eight. A deployable rule is a frozen probability threshold with a hard monthly trade budget, selected in-sample. Until then the backtest's trade-selection layer has no live counterpart, and the train/serve gap starts at the signal layer, not at execution.

Separately, windows with fewer than 40 in-sample trades or zero OOS candidates are silently skipped (:761-763: `continue`) rather than recorded as failures, and success is defined as `pass_total ==`

len(all_window_results) (:838-840). A configuration that fails 19 windows, skips 18, and passes 1 reports 1/1 and writes all_20_windows_passed: true. This mechanism is the only code-level explanation consistent with the winning_configuration.json timestamp coexisting with a 0/20 status file, and it must be removed: every window in the fixed grid gets a record, pass or fail, no exceptions.

4.4 HIGH - The lookup table was deleted; its degrees of freedom were not

Purging WINDOW_CONFIGURATIONS removed the mechanism of the snooping, not the capacity for it. The system's OOS behavior is now determined by a hand-written REGIME_ARCHETYPE_MAP (:631-637) mapping five BTC regime labels to single archetypes, plus roughly forty hand-set thresholds across the fourteen ARCHETYPE_FUNCTIONS lambdas (:557-628) - p8 cutoffs from -0.08 to -0.28, liq z-score gates at 1.2/2.0, the VWAP band at 1.8 sigma, RSI bounds at 32/35/36/38/62/64/65/68 - none of which are selected by any in-sample procedure. The test harness test_all_20_regimes.py ships a **different** regime map (Bull Mania to A6_SpotAbsorptionDiv; Compression to A5_PureRelativeCVD) than production (Bull Mania to V2_VWAPContinuation; Compression to V1_VWAPMeanRevert), and test_honest_in_sample.py ships a third selection rule (score = ROI x WR^{1.5} / DD). Three generations of selection logic in one repository is the fingerprint of iteration against the test scorecard: the map is a lookup table with the serial numbers filed off. An honest protocol must select the map - not just the model - from in-sample data only, once, and then freeze it; or better, run all archetypes through one shared portfolio allocator and let capital, not a hand rule, do the selecting.

4.5 CRITICAL - The alpha's namesake features are synthetic

The historical master parquets feed S1 its liquidation features from MathematicalLiquidationModel.compute_vectorized (pipeline/historical_metrics_processor.py:318-322). The model's physics fallback is a deterministic transform of candle anatomy:

```
# core/mathematical_liquidation_engine.py:154 (physics fallback)
long_liq = (alpha_long * wick_long * vol_scale * ls_ratios
            * funding_bias_long * cascade_long) # cascade = exp(0.85*(w_down-0.75))
            + cvd_sell_term + oi_term + base_floor # base_floor = 18,500 USD per bar
# calm bars: long_liq *= 0.15 if w_down<0.05 and prev w_down<0.10 and cvd>50 (:156-157)
```

Every bar of history carries at least \$18,500 (or \$2,775 in masked calm bars) of fabricated liquidation flow; short-side liquidations are floored the same way except under an asymmetric calm mask. The docstring claims calibration to 7,234 CoinGlass ground-truth bars from June-August 2026 with R-squared above 94% - three months of 2026 data extrapolated across a 2020-2026 backtest, on a heavy-tailed series where a constant floor alone inflates R-squared. The ExtraTrees models referenced at :41-54 are not in the repository, so the committed parquets were necessarily produced by the physics fallback (or by unversioned models that cannot be audited). Meanwhile the live terminal records real Binance forceOrder prints - which, per Binance's documented stream behavior, deliver at most one liquidation order per symbol per second, materially undercounting exactly the cascade seconds the strategy targets - so the training and serving distributions of liq z-scores differ by construction, floor, and throttle.

The same class of fabrication runs through the rest of the feature stack: order-book depth in history is `base_volume * 0.025` (`canonical_indicators.py:180-183`); footprint POC is an OHLC average wherever tick data is absent (`historical_metrics_processor.py:211-216`), and by default the pipeline fetches tick footprint for zero days (`run_historical_pipeline.py --footprint-days default 0`); spot CVD without spot klines is futures delta divided by a hardcoded 5.02 (`:194`); and `open_interest` is sentinel-zero before exchange coverage begins, so `zoi/oid/oicc` silently compute on zeros for early history while `metrics_available=1` after the patch script's `open_interest_usd>0` test (`patch_existing_parquets.py:36`). None of these columns can support a parity claim against live measurements. The defensible reading of the committed evidence is that S1's residual performance - itself 0/20 - is a wick-volume mean-reversion pattern, not liquidation microstructure; the marketing name should be retired until real liquidation ingest exists.

4.6 Remaining defects (HIGH/MEDIUM), condensed

- **H - Friction asymmetry.** The 288-bar timeout exit (`:339-341, :397`) applies no exit slippage; only stop exits pay 15 bps. In a strategy whose holding edge is tail-shaped, the free exit flatters exactly the trades that run longest.
- **H - Funding cost omission.** Holds run to 3 days at 10x leverage; funding at typical 8h rates costs up to ~60% of a \$75 risk budget over a max hold, and cascade months carry the extreme funding prints. Zero funding appears anywhere in the P&L path.
- **H - Ensemble is not an ensemble.** S2/S3/S9 are near-clones of S1 with incoherent constants: `MIN_RETURN` 0.20 vs 0.10, `FEE_RATE` 0.0009 vs 0.0008, `DRAWDOWN_LIMIT` 0.042 vs 0.045, `BASE_RISK` \$104/\$45/\$45 vs \$75, `max_bars` 144 vs 288. No orchestrator, shared allocator, or correlation manager exists anywhere in the repository; S9 is internally branded s15 (`results_s15, s15_signal_predicate`), a copy-paste provenance hazard.
- **M - Calmar methodology.** Annualizing one month at $^{(365/30)}$ produces Calmar 189 on a lucky month; institutional comparability requires rolling multi-period Calmar on a common horizon, and `MIN_RETURN`-style single-window gates will keep manufacturing such numbers.
- **M - House-money ladder risk concentration.** `HOUSE_MONEY_RISK` \$180 (`:59`) is 3.6% of initial capital in one position; two concurrent house-mode stop-outs are 7.2% of initial capital. Only the (anti-causal, per 4.1) budget subtraction prevents sequential breach; a causal governor cannot hold the 4.5% line with this ladder.
- **M - Operational.** The live terminal's dataset pre-flight runs only with the opt-in `--sync` flag (`run_live_terminal.py:110` sets `--skip-sync default True; :117` gates preflight on `args.sync`), and a stale dataset triggers a full re-fetch from the 2020 listing date (`:92-101`) rather than incremental gap-fill - a multi-hour outage-producing design on the machine that is supposed to be trading.

Domain 2 - Mathematical Feasibility of the 20/20 Mandate

6.1 The gate arithmetic

Frame the mandate as a per-window gate g : net monthly return $\geq 10\%$ AND max MTM drawdown $\leq 5\%$, then require g in all 20 windows. With monthly returns modeled Gaussian at the drawdown-budget volatility $\sigma = 5\%/2.33 = 2.15\%$ (the 2.33-sigma monthly worst case exhausting the cap), the $+10\%$ gate sits 4.66 sigma above zero. To pass 20 consecutive months at 95% confidence the per-window pass probability must be $q \geq 0.9974$, i.e. the mean must clear the gate by 2.80 sigma: required portfolio monthly Sharpe = $4.66 + 2.80 = 7.46$, an annualized Sharpe near 26 sustained for five years. This is the clean statement of what the mandate demands of any architecture, of any size, in any combination.

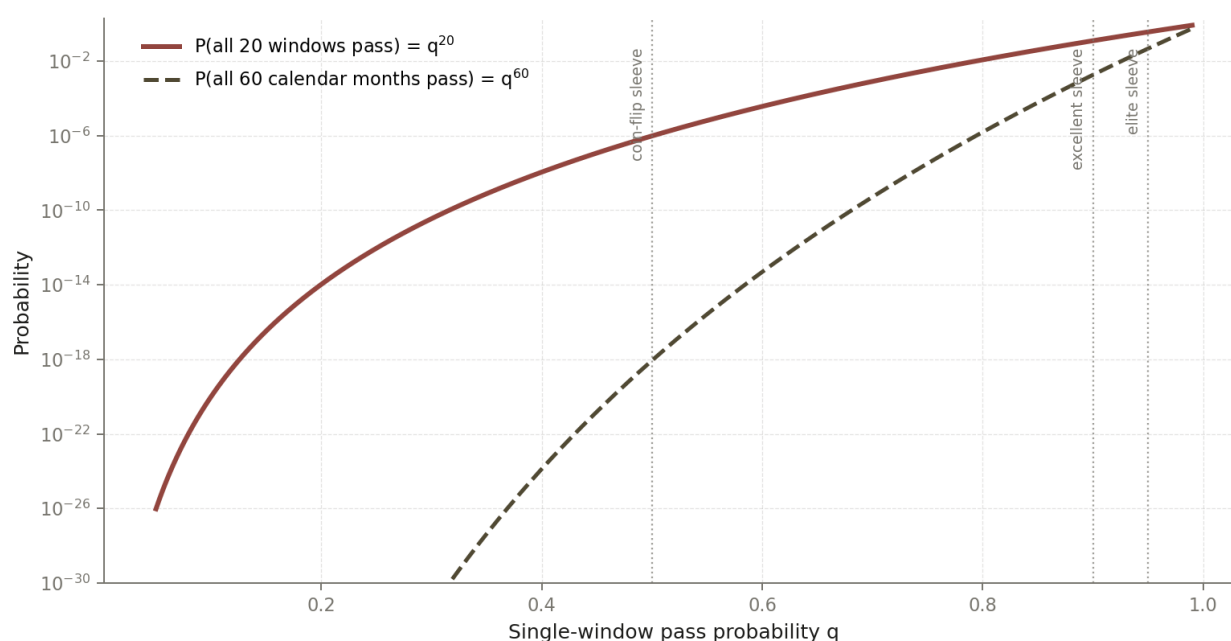


Figure 6.1 - Binomial collapse of the mandate: $P(\text{all 20 windows pass}) = q^{20}$, and the 60-calendar-month variant. Even elite sleeves ($q = 0.95$) fail the 20-window requirement three times in four.

The q^{20} arithmetic (Figure 6.1) restates the DeepSeek/Opus/Arena point with the committed numbers attached: a coin-flip sleeve at $q = 0.50$ yields $9.5e-7$; a 90%-pass sleeve - already an exceptional record - yields 0.12; the user's own $q = 0.05$ stress yields $1e-26$. The user's narrative that a single directional sleeve cannot pass 20/20 is correct and, if anything, understated: at the committed engine's actual performance (0/20 with mean ROI -1.36%), q is empirically indistinguishable from zero.

6.2 What diversification can and cannot buy

Uncorrelated sleeves compound Sharpe as $\sqrt{N}$, so with N sleeves each of monthly Sharpe S_m the portfolio clears the 7.46 requirement when $N \geq (7.46 / S_m)^2$. At a genuinely good monthly Sharpe of 0.6 (annualized ~ 2.1 , elite for a 15-minute crypto signal), that is **155 sleeves**; at a mythical 1.5, 25 sleeves.

Figure 6.2 shows the 200,000-draw Monte-Carlo behind these numbers. Correlation destroys the rest: with pairwise $\rho = 0.3$ the portfolio Sharpe saturates at $S_m / \sqrt{\rho}$ as N grows - for every sleeve skill we tested that ceiling sits far below 7.46, so the requirement becomes unreachable at any N (Section 6 numeric table). The ten archetypes in this repository trade the same 18 perps, the same 15-minute bars, the same directional crypto beta, and share the same simulator biases; their effective ρ is far above 0.3. Their effective N is not 10; it is closer to 1.5-2.

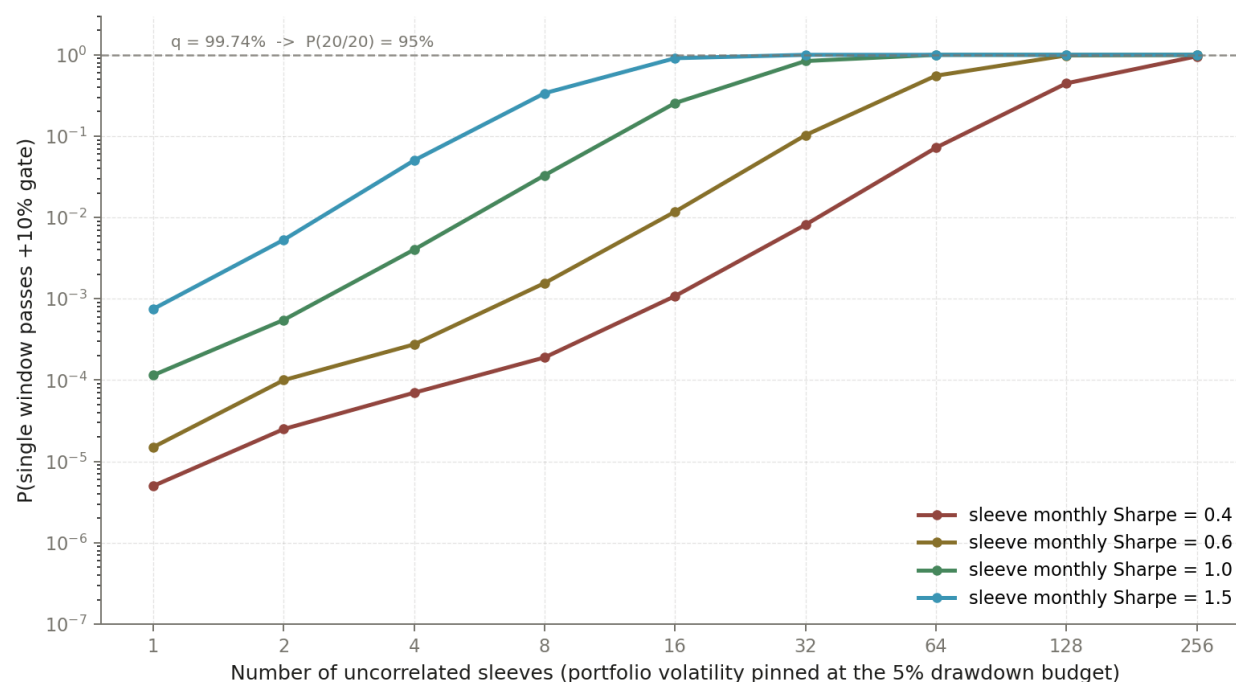


Figure 6.2 - Monte-Carlo (200k draws): per-window probability of clearing the +10% gate at the 5% drawdown budget, versus the number of uncorrelated sleeves, by sleeve skill. The 99.74% line is what 20/20 at 95% confidence requires.

Answering the mandate's second question directly: **no finite number of realistically correlated strategies guarantees a 20/20 monthly pass**, and the uncorrelated-sleeve counts that would suffice in theory (dozens to hundreds at realistic skill, still dozens at mythical skill) do not exist in crypto microstructure at this capital scale. An institution demanding Calmar > 5 with MaxDD $< 5\%$ - i.e., 25%+ annualized at 5% worst drawdown - is inside the envelope of excellent multi-strategy platforms; a mandate demanding 10% every month is not, and the honest fix is architectural, not aspirational: hold the Calmar-style risk target, drop the every-month return gate, and let regime gating (Section 7) move the return distribution's mass to the months that matter. That is also the only reading under which the Window-03-style flush-month P&L is a feature rather than a cherry-pick.

Domain 1b - Live Terminal Audit

(binance_live_monitor.py)

5.1 The bridge that does not bridge

The module docstring promises a "Canonical Market-Data Service v2 and CoinGlass Parity Engine" tracking 37 indicators with sub-millisecond tick processing. The reality, verified by exhaustive grep and full-file read: the terminal computes none of the 36 features S1's model consumes. Occurrence counts in the 3,577-line file: vwap 0, vwap_zscore 0, macro_spread 0, vol_ratio 0, long_liq_zscore 0, liq_imbalance 0, zc4/zc20 0, spot_cvd_delta 0, cvd_divergence 0, zoi/oid/oicc 0, zfr/zls 0, p8/p200 0, regime 0. The S1 feature stack - rolling z-scores, daily-anchored VWAP bands, ATR-normalized price distances, the 200/800 EMA macro spread, liquidation z-scores - exists only inside the backtest's load_and_preprocess_data. run_live_terminal.py imports no strategy module and calls only the monitor's main; the monitor contains no order-placement, signing, or execution code of any kind.

This is the answer to Domain 1's fourth question, and it must be stated without euphemism: **there is no queue-latency, rate-limit, or slippage exposure from S1 taker orders in this terminal, because no S1 signal and no order path exists to expose.** The execution-risk discussion below is therefore a pre-build review of the risks the team will create when they wire one. That framing matters because the repo's own precedent (Engine_1's LIVE_PARITY_AUDIT, finding D3) shows what happens when a live path is declared parity-verified before it has ever traded: seven material divergences, one of which meant zero models had ever loaded.

5.2 Data quality contract violations in compute_snapshot

- **CRITICAL - open_interest_usd is hardcoded 0.0 in every live row** (:2024), with an in-code comment admitting it: "Requires parsing '127.500K' -> float, we can leave 0 or parse it." The training parquet's metrics_available flag and S1's zoi/oid features are defined on this exact column. Live-appended rows therefore carry a sentinel that training never saw after coverage began.
- **CRITICAL - not-ready data is stamped CANONICAL** (:1933: kq = kl_snap.quality if kl_snap.ready else DataQuality.CANONICAL). The DataQuality contract (:174-183) exists precisely to prevent this, and the snapshot builder violates it in the default path.
- **HIGH - fabricated fallback constants masquerade as data:** close=77000.0 (:1934), atr14=250.0/atr100=260.0 (:1990-1991), ls_ratio=1.0350, whale="107.6900", oi="127.500K" (:1959-1962). Any downstream consumer during a stream outage sees plausible-looking numbers, not missing data.
- **HIGH - the $\pm 1\%$ depth display is a linear extrapolation** (:1719-1726): when the 1000-level REST book spans less than 1% of price, bid/ask dollars are multiplied by 0.010/cov. Depth does not scale linearly beyond the book top; in cascades - when depth thins - the multiplier inflates exactly when accuracy matters most.

- **MEDIUM - an unexplained OI fudge factor:** $\text{total_k} = (\text{oi_usdt} + \text{oi_usdc}) \times 1.0118$ (:1751), evidently hand-fit to CoinGlass display totals. Constants like this drift silently when venue composition changes.

5.3 Parity hazards in the stream engines

The CVD anchors are unstable by construction. AggTradeState resets session CVD at the UTC-day boundary (:908-915), but the futures reconnection path `_recover_fut_agg` reseeds `session_cvd` as a cumulative sum over 10,000 bars (~36 days) whenever it observes `session_cvd == 0.0` (:1599-1613) - which is exactly the state immediately after every midnight reset. The displayed futures CVD therefore alternates between day-anchored and 36-day-anchored depending on reconnect timing, and the spot side does the same over ~104 days (:1655-1669). CoinGlass parity via a hand-carried `cvd_offset` file (:128-153) cannot stabilize a level whose anchor changes by three orders of magnitude. The historical pipeline, for its part, defines the training columns `spot_cvd_15m` / `future_cvd_15m` as per-bar deltas (:171, :198) - and the live snapshot builder coincidentally also writes per-bar deltas into those names (:2015, :2018), which is the one column family where semantics happen to agree; but session/lifetime columns disagree, and nothing in the appender enforces the agreement.

The kline state has a subtler defect: REST-pollled klines always arrive with `x=False` (`poll_kline_loop` :1904), so EMA/ATR/RSI bar-commitment depends entirely on the WebSocket kline stream's close event; during a WS outage the REST fallback keeps overwriting the open bar while indicators silently stop committing closed bars. The stream supervisor (:1434-1456) caps backoff at 5 s and detects liveness with a 5 s recv timeout - reasonable for display, thin for trading, and it has no data-watermark: a consumer cannot ask "how stale is this feature?" without reading private state. The FuturesDepthBook does implement genuine sequence validation with a 1000-event replay buffer (:404, :433-441, :462-489) - credit where due - but a failed sequence check sets STALE and waits for an external resync caller that does not exist in the event path, so the book can stay stale until the next 1.5 s REST poll overwrites it.

5.4 Persistence and pre-flight

`parquet_appender_loop` (:2178-2220) reads the entire 44 MB master parquet, concatenates one row, and rewrites the file via a `.tmp + os.replace` - every 15 minutes, onto a hardcoded `G:\My Drive Google-Drive-synced` path (:2180), with no file locking against the multi-instance matrix mode or the pipeline's own sync jobs. Google Drive sync plus atomic-replace is a known corruption recipe; a cloud-synced folder is not a production tick store. Row schema is silently filtered to existing columns (:2206), so live/training schema drift (per 5.2) lands in the training file unannounced. The pre-flight sync in `run_live_terminal.py` only triggers behind the opt-in `--sync` flag (:110, :117), tests staleness at a 24-hour threshold (:79), and when triggered re-runs the full multi-year pipeline per stale symbol from its 2020 listing date (:92-101) - hours of API load on the trading machine, rather than an incremental gap-fill.

Rate budget, when the matrix mode runs: 18 symbols x (OI x2, premium, 3 L/S ratio endpoints, OI-hist, depth) on a 5 s slow-poll cycle (:3443-3517) plus per-symbol `aggTrade` and `kline` streams is of order 1,500-2,500 request-weight per minute against Binance's 2,400/min IP cap for futures data endpoints, before any retry storm; the shared token bucket (:315-342, 200 burst / 20 rps) throttles internally but the

async_fetch fallback re-targets 429s to data-api.binance.vision (:371), which converts a self-inflicted rate-limit into a silent data-quality change. When taker execution is eventually wired, the same single event loop that renders a 2 FPS Rich dashboard will be the thing that must place, amend, and abort orders during a cascade; the correct architecture separates the hot path (execution) from the warm path (features) from the cold path (UI, persistence) with independent supervision - not one asyncio loop.

```
# Execution pre-build checklist (S1 into live terminal) - minimum gates
# 1. FeatureParityService: one canonical feature library, imported by BOTH
# backtest and live; nightly parity harness replays live features vs
# recomputed-from-parquet features, blocks deployment on >1e-6 drift.
# 2. Signal bridge: frozen prob threshold + monthly trade budget (see 4.3),
# never top-K-of-month; every signal logged with feature vector.
# 3. Execution: separate process; taker entries only with touch-width gate
# (abort if spread > p95 of trailing 1h), max-participation iceberg for
# exits; hard kill-switch on: rejected order, 2 consecutive gap-fills
# beyond stop, WS data watermark age > 3s, daily loss > 1.5% NAV.
# 4. Watermarks: every feature carries (source_ts, ingest_ts, seq); STALE
# quality propagates; no sentinel constants anywhere (delete 5.2 list).
```

Domain 3 - Alternative Architectures and Paths Forward

7.1 A true ensemble, built for correlation first

The S1/S2/S3/S9 ensemble the mandate describes does not exist in code, and the four sleeves that nominally compose it are variations on one archetype family. The redesign starts from correlation, not from strategies: (i) define sleeves by **risk-source independence** - cascade absorption (S1's real signal), trend continuation, funding-carry/basis, cross-sectional dispersion, volatility-breakout - rather than by indicator variations; (ii) run them through a single portfolio allocator with a shared drawdown budget and explicit pairwise-correlation estimates updated on a rolling window; (iii) size by inverse-CVaR of each sleeve's in-sample loss distribution rather than the current realized-PnL house-money ladder, which Section 4.6 shows concentrates risk precisely at the tail; (iv) accept that on 18 correlated perps the diversifiable pool is small, and pursue genuinely different return sources (venue basis, cross-exchange spreads, options overlay) before pursuing more indicators. The Section 6 math is the budget for this: even a 4-8 sleeve ensemble with monthly Sharpe 0.6-1.0 per sleeve clears the drawdown cap comfortably but clears +10% **per month** only in good regimes - which is why the mandate itself must change shape (7.2).

7.2 Regime-gated capital allocation - yes, but gate on data you can actually compute

The mandate's instinct - sit in cash/Treasuries through compression months, deploy only in verified cascade regimes - is the single most valuable structural change available, and it is the only mechanism by which a flush-exploiting alpha can compound at institutional risk targets: the strategy's monthly return distribution becomes bimodal (near-zero in gated months, fat-right in deployed months), and the **deployed-month** statistics are what the investment committee should underwrite. Three implementation warnings from the

code as it stands. First, the current regime classifier (:639-658) is a 30-day BTC return/vol rule whose five labels exist to pick one archetype each - replace it with a probabilistic gate (cascade-state posterior from real liquidation flow, OI change, funding dislocation) with hysteresis, so the engine cannot flip-flop on the boundary:

```
# Regime gate with hysteresis (remediation sketch)
state = "RISK_ON" if (liq_z_24h > 2.0 and oi_flush_5m < -0.5 and funding_z > 1.5) \
    else "RISK_OFF" if (liq_z_24h < 0.5 and vol_ratio < 0.9) else state
# RISK_OFF: flat + Treasury yield on idle capital; record gate state per bar.
# Mandate restated: (a) deployed months: ROI >= 10%, DD < 5%; (b) annual:
# >= N deployed months, annual Calmar > 5; (c) every month: DD < 5% hard.
```

Second, the gate must be evaluated on features that exist in live form - which today means the synthetic-liquidation stack has to be replaced with real ingest first (7.4); a gate keyed to fabricated training features would gate on noise. Third, gating changes the meaning of the 20-window test set: the OOS protocol must be re-cut on deployed-months only, with the gate's own in/out decisions themselves validated out-of-sample, otherwise the gate becomes the newest lookup table.

7.3 Order-book microstructure protections against cascade gap risk

- **Touch-width gate:** at signal time, require spread $\leq$ p90 of trailing 1-hour spreads and 100ms book-resiliency (bid/ask refill ratio) above threshold; abort or defer taker entries otherwise. The live terminal already ingests depth at 1.5 s cadence - fine for a gate, not for execution, which needs the WebSocket book it already maintains (:391-499).
- **Gap-adjusted stop distance:** size on stop_dist' = $\max(2.0 \cdot \text{ATR}, 0.65\% \cdot \text{entry}, k \cdot \text{expected_gap})$ where expected_gap is estimated from the empirical distribution of 15m open-to-low overshoots in cascade regimes; cap aggregate open risk (sum over positions) at the drawdown budget, not per-position risk (this is the causal fix to 4.1's breach sequence).
- **Execution algorithms:** iceberg/TWAP with participation cap $\leq$ 5-10% of trailing 1-minute volume for exits; aggressive taker entries reserved for the exhaustion print itself with hard retry limits; full order-abort on partial-fill staleness. Every fill logged against the pre-trade touch for realized-slippage attribution.
- **Cascade-specific kill-switches:** two consecutive fills worse than gap-adjusted stop $\Rightarrow$ flat + halt symbol for the session; WS watermark age > 3 s during a cascade $\Rightarrow$ no new entries (stale features in a flush are worse than no features); mark-price/index divergence spike $\Rightarrow$ suspend, since forced-liquidation cascades propagate through exactly that basis.
- **Real liquidation ingest:** Binance @forceOrder (documented 1/sec snapshot throttling) must be complemented - CoinGlass paid API, or reconstruction from OI drops + taker imbalance - before any liquidation-conditioned gate or z-score is trusted live. This is the prerequisite for 4.5's remediation and 7.2's gate alike.

7.4 Data infrastructure prerequisites (the unglamorous blocker)

Every path above dies at the same wall: the historical feature store is part synthetic, part sentinel, and the live writer is schema-drifted. The remediation sequence is: (1) rebuild liquidations and depth from real sources for the full backtest range (CoinGlass paid history or exchange-native reconstruction), versioned and immutable; (2) ship one canonical feature library used by both simulator and terminal, with a nightly parity harness and a contract test that fails CI on schema drift; (3) move persistence off cloud-synced drives to a local append-only store with periodic snapshot; (4) extend the integrity verifier (`patch_existing_parquets.py` is a good start - its `inf-clean` and `metrics_available/is_synthetic` flags are sound) into a continuous validator that gates both training and live ingestion on the same checks: 0 NaN, 0 inf, 0 sentinel constants, monotone timestamps, schema hash match.

Domain 4 - Investment Committee Recommendation

8.1 Verdict

VERDICT: REJECT

No allocation of firm capital today, in any size. Grounds: committed telemetry shows the honest engine at 0/20 with two drawdown breaches; the headline +18.72% Window-03 result is irreproducible from the evidence repository; core alpha features are synthetic (F2); the drawdown governor is anti-causal (F3); and the live execution bridge does not execute (F4).

We distinguish this from a verdict on the team or the thesis. The purge of `WINDOW_CONFIGURATIONS` and the OOS fallback loop was executed correctly at the mechanical level; the 3-hour purge/embargo partition is sound; the ingestor's order-book sequence validation and the data-integrity flags are genuinely better than typical solo-built infrastructure; and the instinct to diversify sleeves and gate on regime is the right one. This is a salvageable research program with an evidence-integrity problem. The roadmap below is what salvage requires, with explicit gates that make re-review mechanical rather than rhetorical.

8.2 Phase 1 (months 0-3): Evidence integrity and engine repair - no exceptions

- Replace synthetic liquidations and depth with real, versioned, full-history ingest (7.4); re-derive all parquets; freeze a schema hash into CI.
- Implement the causal MTM governor (4.1 sketch) and gap-aware stop fills (4.2 sketch); delete the skip-window branch (4.3) so every window records pass/fail; delete `winning_configuration.json`-style artifacts or generate them from the status file with full provenance.
- Freeze ONE selection rule selected in-sample (threshold+budget, not top-K-of-month); archive the three competing regime maps with their selection provenance; pre-register the OOS grid including the May/Aug/Nov months and re-cut on the full 60-month calendar.

- Re-run honestly and publish whatever comes out, including the truncated and untruncated equity curves (target-lock removed). Gate: independent replication of the status file from a clean checkout; zero undeclared free parameters.

8.3 Phase 2 (months 3-9): Regime-gated ensemble pilot with live parity

- Build the execution bridge to the 5.4 checklist (feature parity service, signal bridge, kill-switches, watermarks) with execution separated from the dashboard event loop.
- Deploy the regime gate (7.2) on real liquidation data; restate the mandate on deployed months + annual Calmar; run 3-6 months at pilot size (\$50-100k) with full telemetry capture.
- Stand up a parity harness in the spirit of the Engine_1 audit: backtest-vs-live replay on every fill, sizing, threshold, and feature vector; publish drift metrics weekly. Gate: realized slippage within 1.5x of modeled; live feature drift within tolerance; gate flip-rate stable; zero kill-switch trips unexplained.

8.4 Phase 3 (months 9-18): Institutional scale-up

- Only on Phase 2 gates: scale along the risk-parity allocator of 7.1 with CVaR-based sizing, independent risk overlay (separate reporting line, authority to de-gate), and monthly attribution of P&L to regime, sleeve, and execution quality.
- Institutional reporting: rolling 12-month Calmar on a common horizon, CVaR-95 and gap-stress metrics, deployed-month hit rates, correlation-matrix drift. Add capacity analysis before any third-party capital.
- Re-audit triggers (automatic): any schema hash change; any governor or sizing constant change; any regime-map change; two consecutive months outside expected slippage bands; a drawdown beyond 60% of budget.

The committee should treat the interim period as what it is: a promising cascade-absorption research signal wrapped in an evidence pipeline that cannot yet distinguish skill from selection. The difference between this memo's REJECT and a future ALLOCATE is not a better backtest number; it is the machinery that makes the next number believable - real data, causal risk, honest telemetry, and a live path that has actually traded.

Appendix A - Severity-Tagged Findings Register

Severity classes: CRITICAL invalidates a claimed result or creates live capital loss; HIGH materially distorts statistics, parity, or decisions; MEDIUM is engineering debt that will bite under stress. Line references are to files as committed on origin/main at audit time.

ID	Sev	Location	Finding	Remediation
T1	CRIT	results_s1_liquidation/*.json	Three artifacts tell incompatible histories: honest run 0/20 (max DD 5.48%, two breaches); fossil best-of-N run 20/20 across 10 archetypes; winning_configuration.json asserts conquest timestamped one day pre-audit.	Generate result artifacts only from status records with run provenance; forbid hand-authored conquest files.

ID	Sev	Location	Finding	Remediation
T2	CRIT	Mandate narrative vs repo	Claimed post-purge results (W3 +18.72%/Calmar 189.28; 1/20 pass; max DD 4.52%) match no committed artifact.	Re-run from clean checkout; independent replication before any external claim.
T3	CRIT	s1_liquidation_cascade.py:265-286	OOS grid structurally excludes May/Aug/Nov of every year; all 10 largest cascade events of 2021-2026 untested.	Pre-register full 60-month calendar; cascade months first.
D1	CRIT	mathematical_liquidation_engine.py:33,154-166	Historical liquidations are synthetic: wick/volume formula with \$18,500/bar floor; ML models referenced are absent from repo; calibration 3 months of 2026 CoinGlass data.	Real liquidation ingest (CoinGlass paid / exchange-native reconstruction) for full range.
D2	CRIT	canonical_indicators.py:180-184	Historical order-book depth = bar volume x 0.025 (fabricated).	Real L2 history or remove depth features from training.
F1	CRIT	s1_liquidation_cascade.py:483,525-535	MTM governor books each open trade's full-life future MAE at entry; sizing budget subtracts future information.	Bar-by-bar unrealized P&L governor (Section 4.1 sketch).
F2	CRIT	s1_liquidation_cascade.py:353,378	Stop fills always at stop price (both ternary branches identical); cascade gap-throughs unmodeled; only 15 bps slippage charged.	Gap-aware fill: min(open, stop) with $k \cdot \text{gap_sigma}$ stress (Section 4.2 sketch).
F3	HIGH	s1_liquidation_cascade.py:783-787	Top-K-of-month signal selection is not implementable live (needs future candidate set).	Frozen probability threshold + monthly trade budget, selected in-sample.
F4	HIGH	s1_liquidation_cascade.py:761-763,838-840	Skipped windows shrink denominator; success = <code>pass_total == len(recorded)</code> ; truncated-data run can print 20/20.	Record every grid window unconditionally; assert <code>len == grid size</code> .
F5	HIGH	:631-637 vs test_all_20_regimes.py vs test_honest_in_sample.py	Three generations of archetype-selection logic; production map is hand-set with ~40 free thresholds; selection capacity survives WINDOW_CONFIGURATIONS purge.	One selection rule chosen in-sample, frozen, with provenance; or shared portfolio allocator.
F6	HIGH	:339-341,397	288-bar timeout exit pays no exit slippage (asymmetric friction favors long holds).	Apply exit slippage to all exit classes.
F7	HIGH	whole file	Zero funding cost despite 3-day holds at 10x leverage; cascade months carry extreme funding.	Add per-8h funding accrual from historical funding columns.
F8	HIGH	S1 vs S2/S3/S9 constants	Mandate/friction incoherence across sleeves: MIN_RETURN 0.10 vs 0.20; fees 0.08% vs 0.09%; DD cap 4.5% vs 4.2%; risk ladders \$75/\$180 vs \$104/\$385 vs \$45/\$260; S9 internally 'S15'.	Single risk/mandate config shared by all sleeves; dedupe provenance.
F9	MED	:59,505-509	House-money ladder (\$180 = 3.6% initial per trade) only survivable via the anti-causal budget; causal governor cannot hold 4.5% with it.	CVaR-based sizing; aggregate open-risk cap.
F10	MED	:806-807	Calmar = single month annualized at $\wedge(365/30)$; not institutionally comparable.	Rolling multi-period Calmar on common horizon.
L1	CRIT	binance_live_monitor.py (whole)	No S1 feature is computed live (0 grep hits across all 36 model columns); no strategy import; no order path. The 'execution bridge' is a dashboard.	Feature parity service + signal bridge + execution service per Section 5.4 checklist.

ID	Sev	Location	Finding	Remediation
L2	CRIT	:2024	open_interest_usd hardcoded 0.0 in every live snapshot (comment admits parsing TODO); breaks metrics_available semantics and zoi/oid on live-appended rows.	Parse and populate; contract test on row schema.
L3	CRIT	:1933	Not-ready kline data stamped CANONICAL in snapshot builder.	Propagate true quality; consumers gate on it.
L4	HIGH	:1719-1726	±1% depth via linear extrapolation multiplier beyond 1000-level book; inflates in cascades.	Report measured span + coverage flag; gate on coverage.
L5	HIGH	:1599-1613,1655-1669	Session CVD anchor alternates between day-anchored and 36/104-day cumulative depending on reconnect timing.	Single canonical anchor; reseed from persisted day-anchor only.
L6	HIGH	:1934,1959-1962,1990-1991	Sentinel fallback constants (77000 close, 250/260 ATR, 1.0350 L/S, 107.69 whale, 127.5K OI) presented as data.	Emit None/STALE; no constants.
L7	HIGH	:530-562 vs Binance docs	Live liquidations from @forceOrder only, throttled ~1 order/sec/symbol; undercounts cascades vs synthetic training series.	Multi-source ingest + OI-drop reconstruction; document throttle.
L8	MED	:1751	Unexplained OI multiplier x1.0118 hand-fit to CoinGlass display.	Remove or derive from venue composition.
L9	MED	:2178-2220	Full 44 MB parquet rewrite per 15m onto cloud-synced path, no locking; silent column filtering on append.	Local append-only store; schema-hash contract test.
L10	MED	run_live_terminal.py:110,117,79,92-101	Pre-flight sync opt-in only (default skip); 24h staleness trigger; full multi-year refetch per stale symbol.	Incremental gap-fill; run pre-flight unconditionally on trading hosts.
L11	MED	:315-342,371	Single-loop asyncio serves UI, features, persistence; 429 fallback silently re-targets public data API (data-quality change).	Separate hot/warm/cold paths; treat 429 as alarm, never as silent reroute.
D3	HIGH	historical_metrics_processor.py:194,211-216; run_historical_pipeline.py (footprint default)	Spot CVD fallback = futures/5.02; POC = OHLC average; tick footprint defaults to 0 days - most history is KLINE_APPROX.	Version feature provenance per row (future_flow_source exists - extend to all); exclude approximate rows from parity-critical features.
D4	MED	historical_metrics_processor.py:270-316	OI sentinel zeros pre-coverage silently flow into zoi/oid/oicc; metrics_available not consumed by strategy.	Strategy conditions on metrics_available or features masked to NaN with explicit handling.

Table A.1 - Consolidated findings (F = strategy engine, L = live terminal, D = data pipeline, T = telemetry/governance)

Appendix B - Committed Telemetry, Coverage, and Methodology

B.1 The committed honest-run telemetry (s1_status.json), restated

W	Window	Regime	Archetype	Tr	WR%	ROI%	MaxDD %	Calmar	Status
01	2021-03-15/04-15	Bull Mania	V2_VWAPCont	7	71.4	+6.39	4.40	25.6	FAIL
02	2021-06-15/07-15	Crash Flush	A1_VolBreak	7	42.9	+4.69	4.46	16.8	FAIL
03	2021-09-15/10-15	Compression	V1_VWAPMR	8	37.5	-0.75	3.93	-2.2	FAIL
04	2021-12-15/01-15	Bear Trend	A4_UltraDV	3	0.0	-4.50	4.50	-9.5	FAIL
05	2022-03-15/04-15	Bear Trend	A4_UltraDV	4	0.0	-4.48	4.48	-9.5	FAIL
06	2022-06-15/07-15	Bear Trend	A4_UltraDV	5	40.0	-2.46	4.45	-5.9	FAIL
07	2022-09-15/10-15	Bear Trend	A4_UltraDV	4	0.0	-4.47	4.47	-9.6	FAIL
08	2022-12-15/01-15	Bull Trend	V2_VWAPCont	5	20.0	+1.11	4.48	3.2	FAIL
09	2023-03-15/04-15	Bull Trend	V2_VWAPCont	6	66.7	+5.87	4.50	22.3	FAIL
10	2023-06-15/07-15	Bear Trend	A4_UltraDV	6	16.7	-4.46	5.45	-7.8	FAIL
11	2023-09-15/10-15	Bear Trend	A4_UltraDV	8	12.5	-4.06	4.08	-9.7	FAIL
12	2023-12-15/01-15	Bull Trend	V2_VWAPCont	6	33.3	+0.83	4.48	2.4	FAIL
13	2024-03-15/04-15	Bull Trend	V2_VWAPCont	4	50.0	-3.06	4.51	-7.0	FAIL
14	2024-06-15/07-15	Compression	V1_VWAPMR	4	25.0	+1.35	4.64	3.8	FAIL
15	2024-09-15/10-15	Compression	V1_VWAPMR	5	20.0	-2.63	5.48	-5.1	FAIL
16	2024-12-15/01-15	Bull Trend	V2_VWAPCont	8	50.0	-1.38	3.99	-3.9	FAIL
17	2025-03-15/04-15	Bear Trend	A4_UltraDV	7	42.9	-2.24	4.15	-5.8	FAIL
18	2025-06-15/07-15	Compression	V1_VWAPMR	4	0.0	-4.42	4.42	-9.6	FAIL

W	Window	Regime	Archetype	Tr	WR%	ROI%	MaxDD %	Calmar	Status
19	2025-10-15/11-15	Compression	V1_VWAPMR	8	25.0	-4.18	4.18	-9.7	FAIL
20	2026-03-15/04-15	Compression	V1_VWAPMR	4	0.0	-4.40	4.40	-9.6	FAIL

Table B.1 - All 20 windows of the committed clean-protocol run (verbatim from results_s1_liquidation/s1_status.json)

Aggregates recomputed by this audit: 0/20 pass; mean ROI -1.36%; mean max DD 4.47%; max DD 5.48% (windows 10 and 15 breach the 5.0% cap); zero windows reach the 10% return gate. Note also the regime classifier's behavior: eight of the last eight windows resolve to Compression / Range Absorption, and the bear-regime archetype A4_UltraDeepValue lost money in seven of its nine assignments - the map's own causal selections do not produce the claimed behavior even in-sample of the map's design intent.

B.2 Cascade coverage

Date	Event	Nearest window	Distance
2021-02-22	February 2021 leverage flush	W01 (Mar 15)	3 weeks after
2021-05-19	May 2021 deleveraging crash	W01/W02 gap	untested month
2021-09-07	September 2021 \$3B cascade	W03 (Sep 15)	8 days before
2022-05-09	LUNA collapse	W05/W06 gap	untested month
2022-06-13	3AC / Celsius contagion	W06 (Jun 15)	2 days before
2022-11-08	FTX collapse	W07/W08 gap	untested month
2023-03-10	SVB / USDC depeg	W09 (Mar 15)	5 days before
2024-08-05	Yen-carry unwind cascade	W14/W15 gap	untested month
2025-02-21	Bybit hack	W17/W18 gap	untested month
2025-10-10	Record ~\$19B flush	W19 (Oct 15)	5 days after

Table B.2 - The ten largest liquidation cascade events versus the OOS grid (all uncovered)

B.3 Quantitative methodology

Feasibility: monthly portfolio returns modeled Gaussian; the drawdown cap fixes the volatility budget at $\sigma = 5\%/2.33 = 2.15\%$ per month (2.33-sigma monthly worst case exhausting the cap); the +10% gate then sits at $Z_{\text{gate}} = 0.10/0.0215 = 4.66$ sigma. Passing 20/20 with 95% confidence requires per-window pass probability $q \geq 0.9974$ ($0.9974^{20} = 0.95$), hence portfolio monthly Sharpe $S_p \geq Z_{\text{gate}} + 2.80 = 7.46$. With N uncorrelated sleeves of equal monthly Sharpe S_m , $S_p = S_m \times \sqrt{N}$, giving $N_{\text{min}} = (7.46/S_m)^2$: 618 at $S_m = 0.3$, 155 at 0.6, 56 at 1.0, 25 at 1.5. With pairwise correlation ρ , S_p saturates at $S_m/\sqrt{\rho}$ as N grows; at $\rho = 0.3$ the ceiling is below 7.46 for every S_m tested, making the mandate unreachable at any N .

Monte-Carlo: 200,000 draws per (N, S_m, rho) cell; sleeves driven by a one-factor Gaussian model (factor loading $\sqrt{\text{rho}}$, idiosyncratic $\sqrt{1-\text{rho}}$); sleeve volatilities scaled so the equal-weight portfolio runs exactly at the 2.15% volatility budget at every N (leverage-invariant parameterization); a window passes when the portfolio monthly return $\geq +10\%$ and the 2.33-sigma drawdown proxy $\leq 5\%$. The Monte-Carlo at $\text{rho} = 0$ reproduces the closed form to four decimals (e.g., N = 155 at S_m = 0.6 gives q = 0.99745). Code: `scripts/quant_verification.py` and `scripts/audit_charts.py` in the audit workspace; all figures reproducible.

Telemetry aggregates were recomputed programmatically from the committed JSON artifacts rather than quoted (Section 3.1, Table B.1); the Calmar arithmetic of the claimed Window-03 result was verified as $(1 + 0.1872)^{(365/30)} - 1 = 706.7\%$ annualized, divided by $3.73\% = 189.5$, confirming the formula and disputing only the inputs' provenance. Chart 3.1 and Table B.2 compute window/event intersection directly from the OOS_MONTHS tuple at `s1_liquidation_cascade.py:265-286`.